An Introduction to Tiled Parallel Programming, Simplified

The matrix product on the GPU, with a worked example in Rust and cuda-oxide

Arpan Pathak

November 2026

ABSTRACT

In the direct GPU kernel for matrix multiplication, every thread fetches its operands from global memory, so each input value is fetched once for every thread that uses it. Tiling removes most of these fetches: the threads of a block copy a $T \times T$ tile of each input into shared memory, and every thread of the block then takes its operands from the copy. In this tutorial we derive the tiled kernel step by step on a single pair of 4×4 matrices, show why each of its 2 barriers is necessary, prove the kernel correct, and implement it in Rust with cuda-oxide. On an NVIDIA Jetson AGX Thor with 512×512 matrices, the tiled kernel takes 0.285 ms and the direct kernel 1.158 ms, a speedup of 4.06. We explain why the speedup is smaller than the 16-fold reduction in fetches. Appendix B gives a variant in which the GPU accesses the inputs directly in host memory.

KEYWORDS parallel programming education, GPU computing, tiling, shared memory, matrix multiplication, synchronization

1 Introduction

A GPU computes quickly but fetches data from its main memory slowly. Tiling is the standard remedy [2]: a group of threads copies a small block of data into fast on-chip memory once, and then all threads of the group work from the copy. Matrix multiplication is the classic first example, because every input value is needed by many threads and the saving is easy to count.

We assume that the reader programs in Rust and can multiply small matrices by hand. We assume no knowledge of GPUs. Each GPU concept appears at the moment the algorithm needs it, and each step is first worked out on a single pair of 4×4 matrices, small enough to check with a pencil. Only then do we state the general case. In Figure 1.1 we preview where we are going: 2 kernels that compute the same product, but fetch their data from global memory in very different ways.

In Section 2 we define the matrix product and the layout of a matrix in memory. In Section 3 we introduce the GPU: threads, blocks and grids, the memories of the device, and the barrier. In Section 4 we write the direct kernel and count its memory traffic. In Section 5 we derive the tiled kernel, and in Section 6 we explain its barriers and prove it correct. Section 7 gives the Rust implementation of both kernels, and Section 8 our measurements on an NVIDIA Jetson AGX Thor. Appendix A lists the complete program, and Appendix B a unified-memory variant.

We write algorithms in the style of Cormen, Leiserson, Rivest and Stein [1]: procedure names in small capitals, numbered lines, and mathematical notation inside the pseudocode. Braces delimit blocks. The range $[a, b)$ denotes the integers $a, a+1, \ldots, b-1$, exactly like `a..b` in Rust, so $i \in [0, M)$ runs over $0, 1, \ldots, M-1$. Indices start at 0.

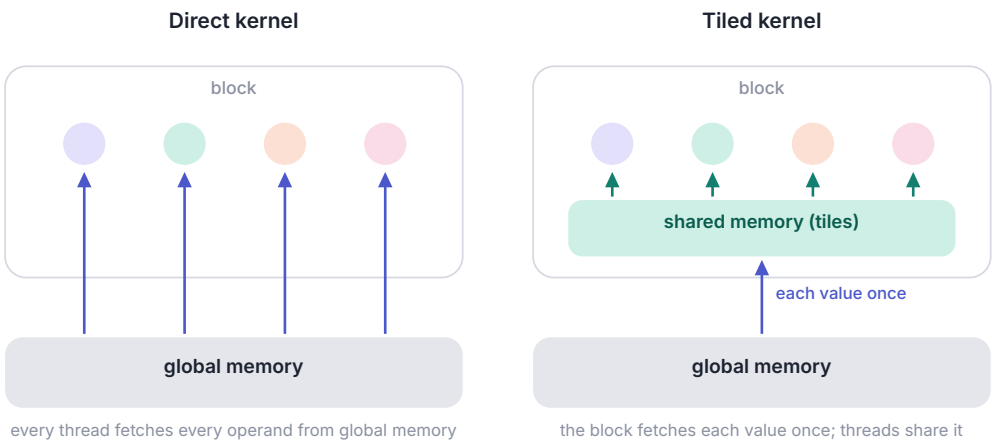


Figure 1.1 The 2 kernels of this tutorial. Left: in the direct kernel, every thread fetches every operand from global memory. Right: in the tiled kernel, the block fetches each value once into shared memory, and the threads of the block share the copy.

2 The matrix product

2.1 A worked example

We use the same pair of matrices throughout the tutorial. Here are both factors and their product, side by side:

$$\underbrace{\begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix}}_{A} \times \underbrace{\begin{pmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ 1 & 2 & 1 & 0 \end{pmatrix}}_{B} = \underbrace{\begin{pmatrix} 11 & 13 & 8 & 8 \\ 27 & 29 & 24 & 24 \\ 43 & 45 & 40 & 40 \\ 59 & 61 & 56 & 56 \end{pmatrix}}_{C}.$$

Each entry of C comes from 1 row of A and 1 column of B. To get the top-left entry, 11, we walk along row 0 of A and down column 0 of B together, multiply the numbers we meet in pairs, and add the 4 products (Figure 2.1). Every other entry of C is made the same way from a different row and column.

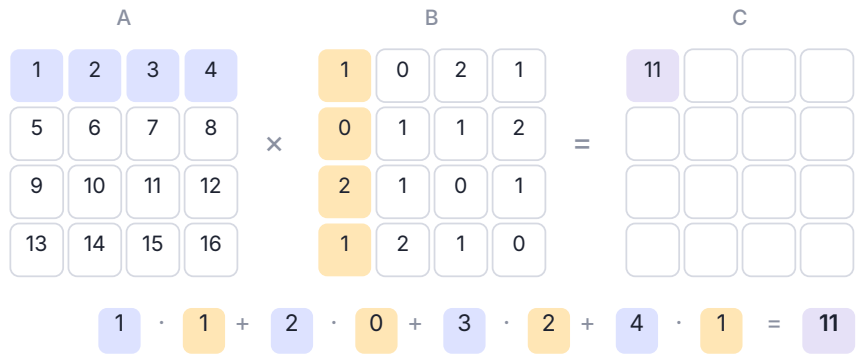


Figure 2.1 The entry c_{00}: row 0 of A paired with column 0 of B. The 4 products sum to 11.

Keep 2 of these entries in mind, $c_{00} = 11$ and $c_{01} = 13$. In Section 6 we will see 2 faulty kernels

produce 7 and 15 in their place.

2.2 Definition

The rule pairs a row of A with a column of B, so a row of A and a column of B must have the same length. We call that length K.

Definition 2.1: Matrix product Let A be an $M \times K$ matrix and B a $K \times N$ matrix. Their product $C = AB$ is the $M \times N$ matrix with entries

$$c_{ij} = \sum_{s \in [0,K)} a_{is}\, b_{sj}, \qquad i \in [0, M),\ j \in [0, N).$$

In words, c_{ij} is the dot product of row i of A and column j of B.

In our example $M = K = N = 4$. For c_{00} the definition gives $1 \cdot 1 + 2 \cdot 0 + 3 \cdot 2 + 4 \cdot 1 = 11$, the sum of Figure 2.1.

2.3 Storage in row-major order

Memory is a single line of numbered locations, so we must decide how to lay a 2-dimensional matrix along that line. The usual choice copies the matrix out the way we read a page: row 0, then row 1, and so on (Figure 2.2).

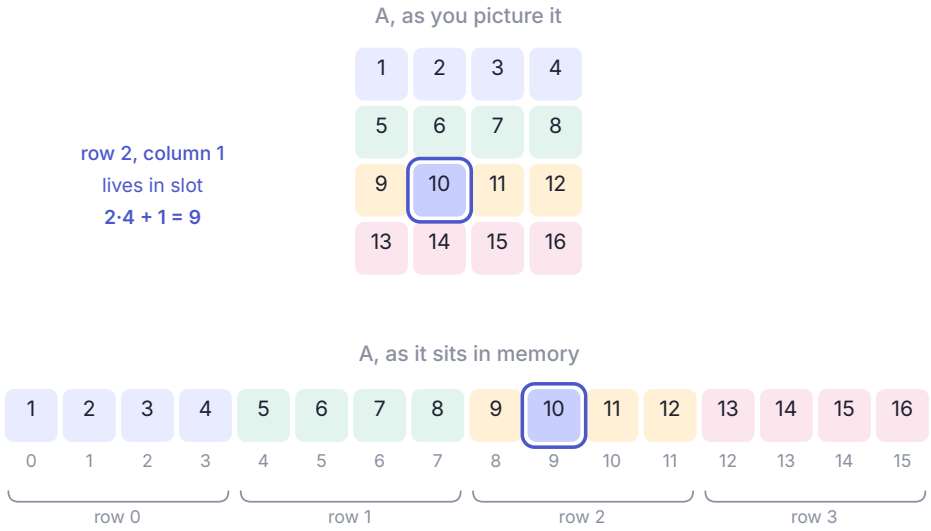


Figure 2.2 Matrix A in row-major order, 1 colour per row. Entry 10, in row 2 and column 1, lands at location $2 \cdot 4 + 1 = 9$.

To find entry (r, c), we skip the r complete rows above it and then the c entries before it in its own row.

Definition 2.2: Row-major order In row-major order the rows of a matrix are stored one after another. In a matrix with rows of width W, entry (r, c) is stored at location

$$\text{Index}(r, c, W) \;=\; r \cdot W + c.$$

A useful consequence: neighbours in a row are neighbours in memory, while neighbours in a column lie W locations apart. Matrix A has rows of width K; matrices B and C have rows of width N.

2.4 The sequential algorithm

On a CPU, Definition 2.1 becomes 3 nested loops. The outer 2 loops choose an entry of C, and the inner loop adds up the products for that entry.

Matrix-Multiply(A, B, C, M, K, N)

```
1  for i ∈ [0, M) {
2      for j ∈ [0, N) {
3          c_ij = 0
4          for s ∈ [0, K) {
5              c_ij = c_ij + a_is · b_sj
6          }
7      }
8  }
```

Line 5 executes once for every choice of i, j and s, that is MNK times. Our example needs 64 multiply-adds; 2 matrices of size 1024×1024 need more than 10^9. The CPU performs them one after another, visiting the entries of C in order (Figure 2.3). A GPU instead performs many of them at the same time.

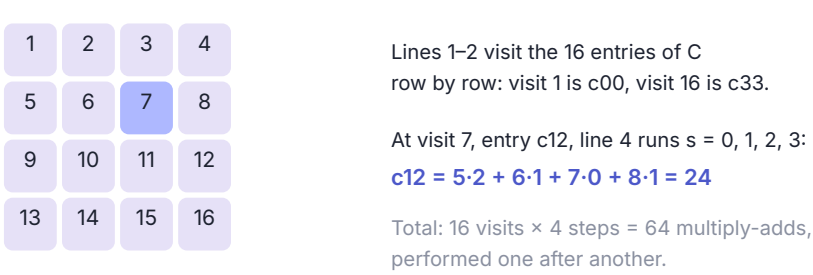


Figure 2.3 The order in which Matrix-Multiply fills C. Visit 7 computes c_{12} with 4 multiply-adds.

EXERCISES

2.1 Compute c_{23} from Definition 2.1. Which row of A and which column of B do you use?

2.2 At which location is b_{31} stored, in row-major order with $W = 4$?

2.3 Show that Matrix-Multiply reads each entry of A exactly N times. How many times does it read each entry of B?

3 The graphics processor

3.1 Host and device

A computer with a GPU contains 2 processors, each with its own memory. We call the CPU and its memory the **host**, and the GPU and its memory the **device**. Data move between the 2 memories only when the program copies them, so every program in this tutorial follows the same 5 steps (Figure 3.1).

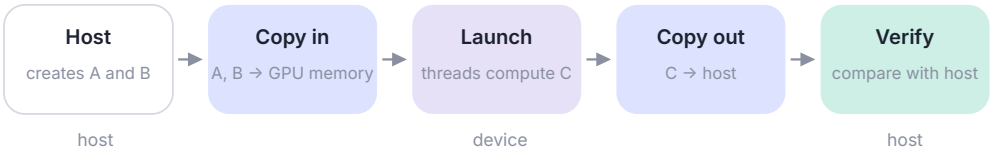


Figure 3.1 The 5 steps of every program. We create the inputs on the host, copy them to the device, multiply on the device, copy the result back, and check it on the host.

Physically, host and device are joined by a PCI Express link (Figure 3.2). In our program, the host holds `a_host` and `b_host`, and the device holds the copies `a_gpu`, `b_gpu` and `c_gpu`.

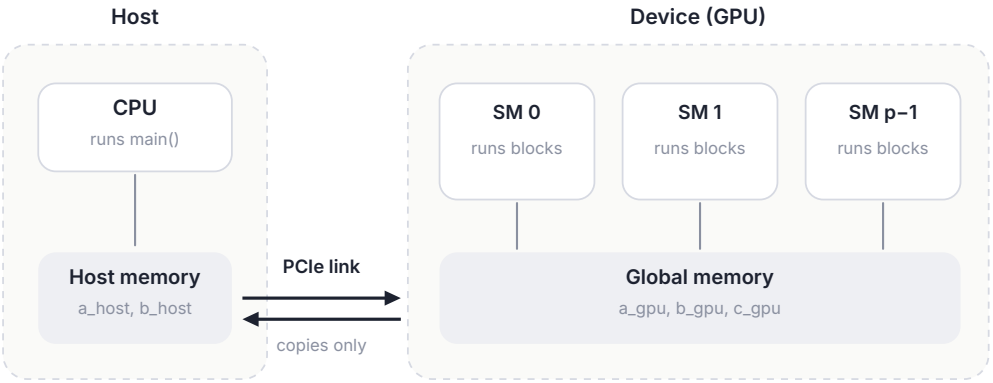


Figure 3.2 The system. Host and device each have a processor and a memory, and the PCI Express link between them carries only explicit copies.

3.2 Threads, blocks and grids

Look again at the 16 entries of C. No entry depends on any other, so we may compute all of them at the same time. A GPU is built for exactly this situation: it runs 1 function many times at once.

Definition 3.1: Kernel and thread A **kernel** is a function that the GPU runs many times at once. Each of these runs is a **thread**; every thread has its own position and its own variables.
Example. We will run the direct kernel of Section 4 with 16 threads, 1 per entry of C; the thread at row 1 and column 2 computes $c_{12} = 24$.

The GPU organizes threads on 2 levels (Figure 3.3).

Definition 3.2: Block and grid A **block** is a group of threads that run on the same processor of the device, share a region of fast memory, and can wait for one another. The **grid** is the set of all blocks of a launch.
Example. With blocks of 2×2 threads, a grid of 2×2 blocks covers C; block $(0, 0)$ computes c_{00}, c_{01}, c_{10} and c_{11}.

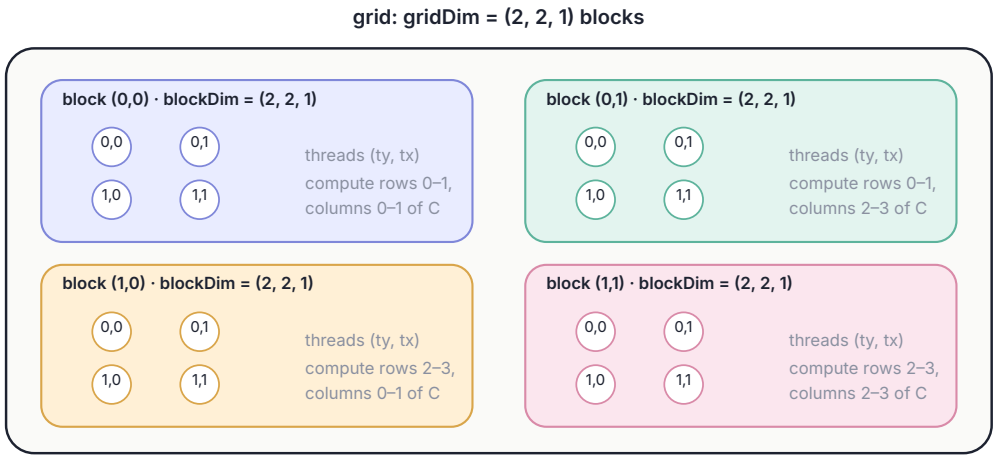


Figure 3.3 The 2 levels in our example: a grid of 2×2 blocks, each of 2×2 threads. Each block computes 1 square of C.

Blocks and grids can extend along 1, 2 or 3 axes, named x, y and z, and we choose the shape to match the data. Every thread can ask the GPU 3 questions: which block am I in (*blockIdx*), where am I inside my block (*threadIdx*), and how big is a block (*blockDim*)? From the 3 answers, a thread can work out which piece of the data belongs to it. We see how, first in 1 dimension.

1 dimension. Take a vector of 12 elements, handled by blocks of 4 threads: 3 blocks side by side (Figure 3.4). Which element belongs to thread 1 of block 2? Blocks 0 and 1 come first and cover elements 0–7, so block 2 starts at element 8, and its thread 1 takes element $8 + 1 = 9$. In general, block b starts at element $b \cdot \mathit{blockDim.x}$, and thread t of that block handles element

$$i = \mathit{blockIdx.x} \cdot \mathit{blockDim.x} + \mathit{threadIdx.x}.$$

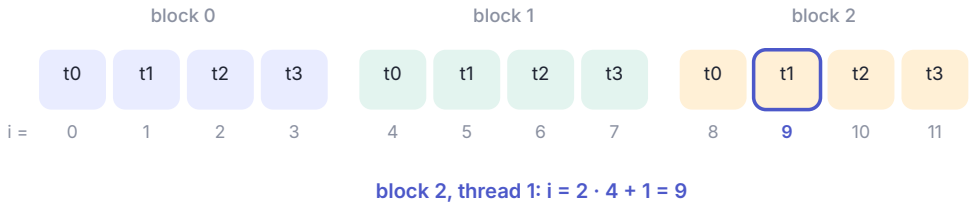


Figure 3.4 1 dimension: 12 elements in 3 blocks of 4 threads. Thread 1 of block 2 handles element $2 \cdot 4 + 1 = 9$.

2 dimensions. A matrix has rows and columns, so we arrange blocks and threads the same way, with x counting columns and y counting rows. For the matrix product we cut C into $T \times T$ squares, where T is the tile width (the constant `TILE` in our code), and give each square to 1 block (Figure 3.5).

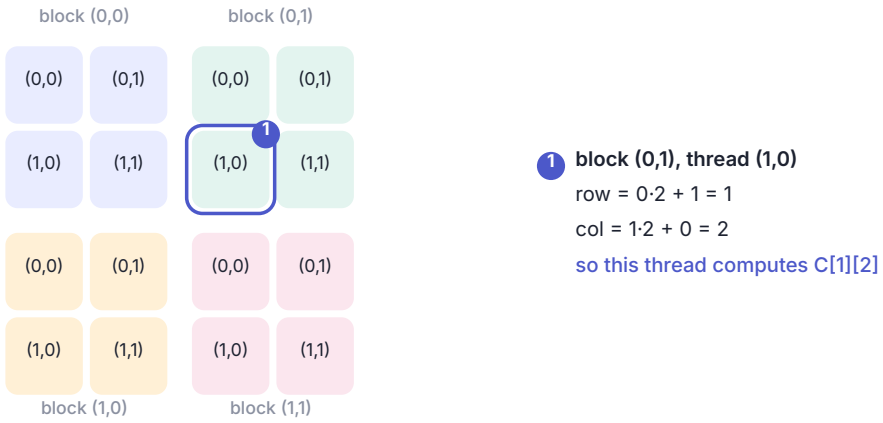


Figure 3.5 2 dimensions: C with $T = 2$, 1 colour per block. Thread positions repeat in every block, so a thread also needs the position of its block.

Which entry of C belongs to the outlined thread? It is thread $(1, 0)$ of block $(0, 1)$, writing positions as (row, column). Its block covers rows 0–1 and columns 2–3 of C, so the block's top-left corner is at row $0 \cdot T = 0$ and column $1 \cdot T = 2$. Inside the block, the thread sits 1 row down and 0 columns across. The thread therefore owns row $0 + 1 = 1$ and column $2 + 0 = 2$, and computes c_{12}. The same reasoning works for every thread:

$$\mathit{row} = \mathit{blockIdx.y} \cdot T + \mathit{threadIdx.y}, \qquad \mathit{col} = \mathit{blockIdx.x} \cdot T + \mathit{threadIdx.x}.$$

In cuda-oxide we get these 2 values from `thread::index_2d_row()` and `thread::index_2d_col()` [4]. Like our Rust code, we shorten *threadIdx.x* and *threadIdx.y* to *tx* and *ty* from here on.

3 dimensions. For data in a volume, such as temperatures on a 3-dimensional mesh, we add the axis z and reason in the same way along each of the 3 axes:

$$x = \mathit{blockIdx.x} \cdot \mathit{blockDim.x} + \mathit{threadIdx.x}, \quad \text{and likewise for } y \text{ and } z.$$

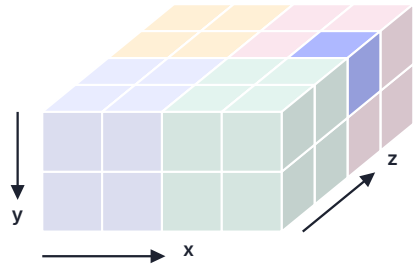


Figure 3.6 3 dimensions: a $4 \times 2 \times 4$ volume covered by 4 blocks of $2 \times 2 \times 2$ threads, 1 colour per block. We compute x, y and z of the highlighted thread separately.

When we launch a kernel in cuda-oxide, we give the sizes of the grid and of a block as triples (x, y, z). For the matrix product we use a grid of $(\lceil N/T \rceil, \lceil M/T \rceil, 1)$ blocks of $(T, T, 1)$ threads.

Scheduling and scope. The device executes blocks on processors called streaming multiprocessors (SMs), which we look at in Section 3.3. At launch, each block is assigned to 1 SM and stays there until it finishes, and an SM can hold several blocks at once. If the grid has more blocks than the SMs can hold, the remaining blocks wait and run in later waves (Figure 3.7). Blocks run in no guaranteed order, so a correct kernel must never depend on the order of blocks.

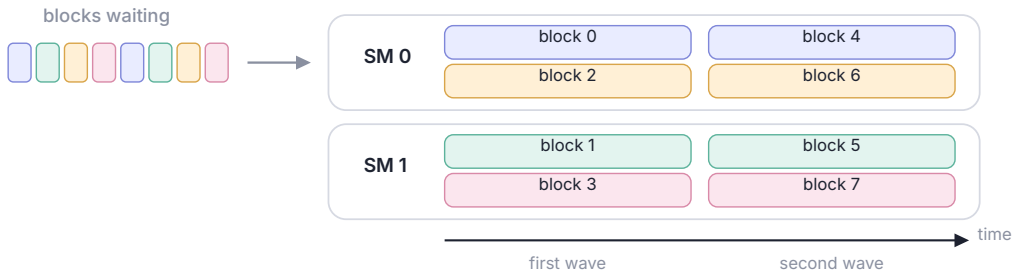


Figure 3.7 8 blocks on 2 SMs that hold 2 blocks each. Blocks 0–3 run in the first wave and blocks 4–7 in the second.

The block is also the unit of cooperation. Threads in the same block share a region of shared memory and can meet at a barrier; threads in different blocks can do neither (Figure 3.8).

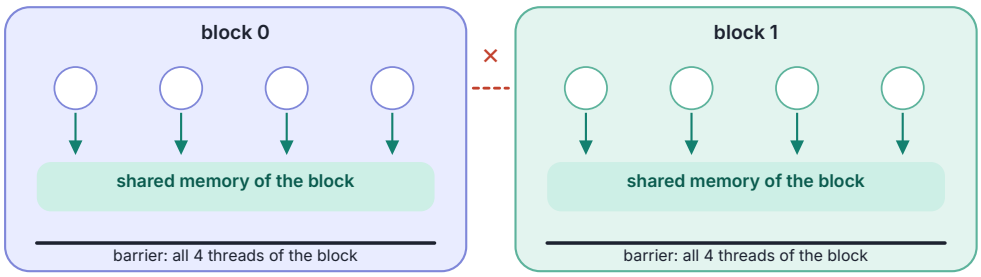


Figure 3.8 Threads of 1 block share shared memory and a barrier. Threads of different blocks can exchange data only through global memory.

3.3 Memories of the device

To compute c_{12}, a thread needs row 1 of A and column 2 of B. How long it waits for them depends on where they are stored, so we now look inside the device (Figure 3.9).

A device contains several dozen SMs. Besides arithmetic units, each SM holds the first 2 of the following memories.

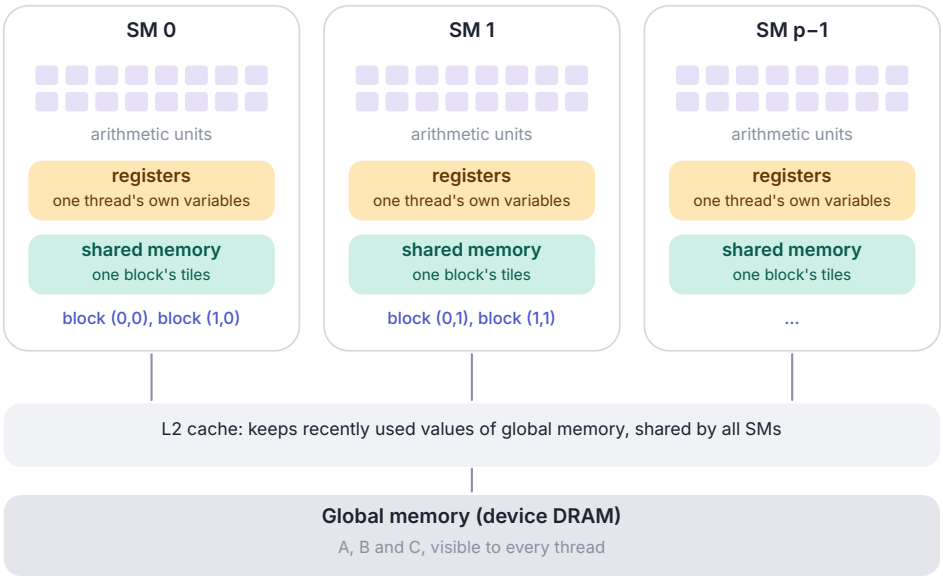


Figure 3.9 The device. Each SM contains arithmetic units, registers and shared memory, and runs whole blocks; the blue labels show 1 possible placement of the 4 blocks of our example. All SMs reach global memory through a common L2 cache.

Definition 3.3: Registers, shared memory, global memory **Registers** hold the variables of a single thread and respond in about 1 cycle. **Shared memory** belongs to a block: every thread of the block can read and write it, no other thread can, it responds within a few cycles, and values enter it only when threads of the block store them there. **Global memory** is visible to every thread, holds gigabytes, and takes several hundred cycles to respond. Between the SMs and global memory sits the **L2 cache**, which keeps recently used values; the program has no control over its contents.

Example. In the tiled kernel, *sum* lives in a register, the tiles live in shared memory, and A, B and C live in global memory.

Storage	Visible to	Latency	Capacity
Registers	1 thread	1 cycle	64K values per SM
Shared memory	1 block	a few cycles	48–228 KB per SM
L2 cache	all SMs	tens to 100 cycles	a few to tens of MB
Global memory	all threads	hundreds of cycles	gigabytes

From the table we can read off our strategy. Arithmetic and registers are cheap; global memory is expensive. We will therefore stage values in shared memory, so that each value crosses from global memory as few times as possible.

3.4 The barrier

The threads of a block do not move in lockstep: 1 thread may run ahead while another falls behind. When threads exchange data through shared memory, they need a meeting point where all of them wait for one another.

Definition 3.4: Barrier A **barrier** is a point in a kernel that no thread of a block may pass until every thread of the block has reached it. In cuda-oxide we write a barrier as `thread::sync_threads()`.
Example. In Figure 3.10, thread 3 arrives first and waits; thread 1 arrives last, and then all 4 threads continue together.

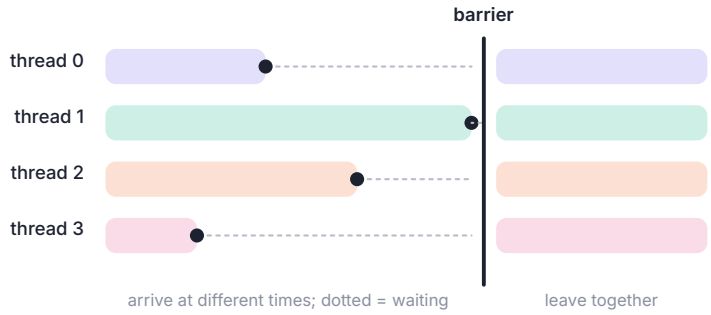


Figure 3.10 A barrier. The 4 threads arrive at different times, wait (dotted), and leave together.

EXERCISES

3.1 Which entry of C does thread $(0, 1)$ of block $(1, 1)$ compute?

3.2 With $T = 16$, how many blocks cover a 1000×1000 matrix C? How many threads in the edge blocks have no entry to compute?

3.3 A grid has 132 blocks, and the device has 66 SMs that each hold 1 block at a time. How many waves are needed?

4 The direct kernel

4.1 The algorithm

We obtain the simplest GPU kernel by deleting the 2 outer loops of MATRIX-MULTIPLY: the position of each thread now plays the role of i and j. Every thread executes the following procedure.

DIRECT-KERNEL(A, B, C, M, K, N)

```
1  tx = threadIdx.x,    ty = threadIdx.y
2  row = blockIdx.y · T + ty,    col = blockIdx.x · T + tx
3  if row ∉ [0, M) or col ∉ [0, N) {
4      return                                   // thread lies outside C
5  }
6  sum = 0
7  for s ∈ [0, K) {
8      sum = sum + a_{row,s} · b_{s,col}        // 2 fetches from global memory
9  }
10 c_{row,col} = sum
```

Lines 3–5 send home any thread that lies outside C, which happens when the matrix size is not a multiple of T. Since every thread writes a different entry of C, the threads never interfere with one another. In Figure 4.1 we follow a single thread through its work.

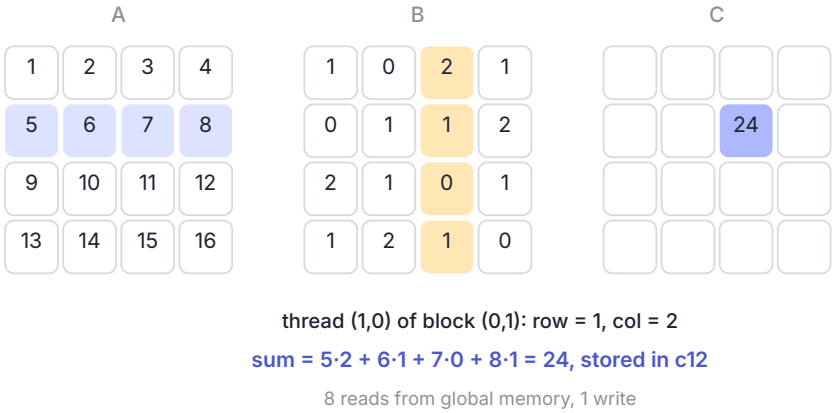


Figure 4.1 The work of 1 thread of the direct kernel: row 1 of A times column 2 of B gives $c_{12} = 24$, with 8 fetches from global memory.

We turn this procedure into Rust in Section 7.1, after we have seen why it is slow.

4.2 Counting the fetches

The kernel is correct, but it is wasteful. Consider the value $a_{01} = 2$. It lies in row 0 of A, and all 4 entries in row 0 of C are built from row 0 of A. So 4 threads need a_{01}, and in the direct kernel each of them fetches it from global memory on its own (Figure 4.2).

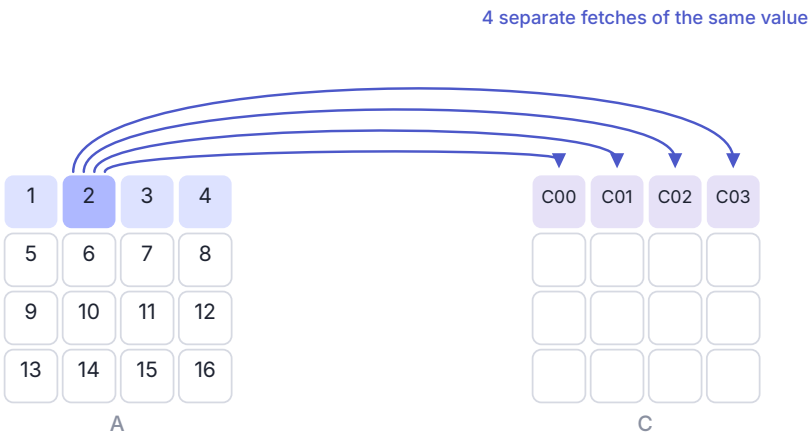


Figure 4.2 The 4 threads that compute row 0 of C each fetch a_{01} from global memory: 4 fetches of the same value.

The same holds for every value. Each entry of A is fetched by the 4 threads of its row of C, and each entry of B by the 4 threads of its column. In total, 16 threads each fetch 4 values of A and 4 of B: 128 fetches, although the 2 matrices hold only 32 distinct values.

In general the direct kernel performs $2MNK$ fetches. For 2 matrices of size 1024×1024, that is about

2.1×10^9 fetches, or 8.6 GB of traffic, to read 8.4 MB of input. Each value is fetched 1024 times, and each fetch costs hundreds of cycles.

KEY OBSERVATION

The 4 threads computing row 0 of C all need the same value a_{01}. If these threads belong to the same block, 1 of them can copy a_{01} into shared memory, and the other 3 can take it from there.

EXERCISES

4.1 Which threads need b_{21}? How many times does the direct kernel fetch it?

4.2 For $M = N = K = 2048$, how many times does the direct kernel fetch each entry of A?

5 Tiling

5.1 Values needed by 1 block

We turn the key observation into an algorithm, starting with block $(0, 0)$, whose 4 threads compute the top-left 2×2 square of C. Together they need rows 0–1 of A and columns 0–1 of B: 8 values from each matrix, 16 in all (Figure 5.1). The direct kernel fetches these 16 values 32 times.

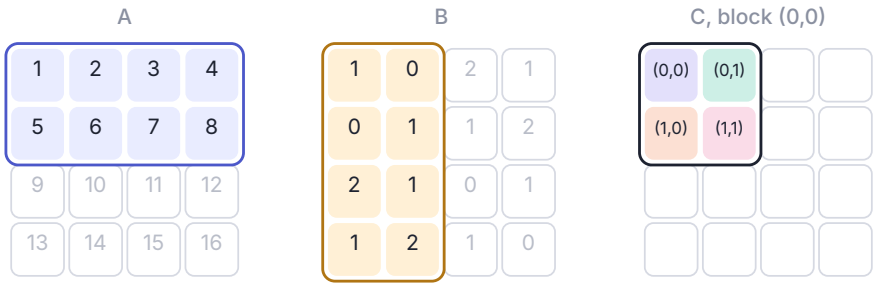


Figure 5.1 The values needed by block $(0, 0)$: 2 rows of A and 2 columns of B.

The plan is clear: the block should copy each of the 16 values into shared memory once, and its threads should then work from the copy. There is 1 obstacle. Shared memory is small. In real problems a row of A may hold thousands of values, while shared memory holds only a few thousand values in total, so a whole strip of rows does not fit.

5.2 Splitting the sum

Fortunately, a thread does not need its whole row and column at once. The entry c_{00} is a sum, and we may add up a sum in any grouping we like. Grouping the 4 terms in pairs gives Figure 5.2.

The first group touches only a 2×2 piece of A and a 2×2 piece of B, and so does the second. The block can therefore work in 2 rounds: in each round it copies 1 small piece of A and 1 small piece of B into shared memory, and every thread adds the matching group of terms to its sum.

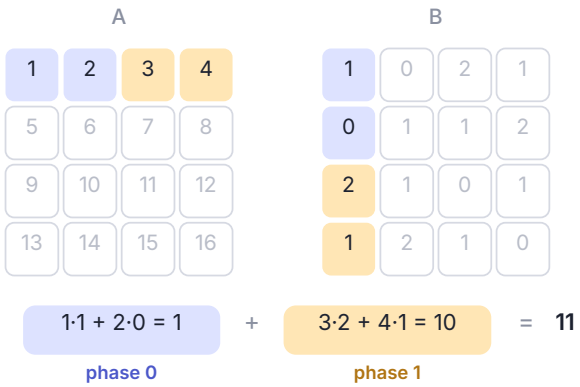


Figure 5.2 c_{00} in 2 groups. The first group (blue) uses only columns 0–1 of A and rows 0–1 of B; the second group (amber) uses only columns 2–3 and rows 2–3.

Definition 5.1: Tile and phase A **tile** is a $T \times T$ piece of A or B that a block holds in shared memory. A **phase** is 1 round of the tiled kernel: the block copies 1 tile of A and 1 tile of B, and every thread adds T terms to its sum.
Example. With $T = 2$, block $(0, 0)$ runs 2 phases. In phase 0 it uses the tile of A in rows 0–1, columns 0–1; in phase 1, the tile in rows 0–1, columns 2–3.

In general we cut the range $[0, K)$ into segments of length T and write each index as $s = tT + u$, where t numbers the segment and u is the position inside it. Definition 2.1 then becomes

$$c_{ij} = \sum_{t \in [0,\, \lceil K/T \rceil)} \underbrace{\sum_{u \in [0,\, T)} a_{i,\, tT+u}\, b_{tT+u,\, j}}_{\text{phase } t},$$

where we treat entries beyond the edge of a matrix as 0, so that a short final segment still adds up correctly. From 1 phase to the next, the tile of A moves 1 step to the right and the tile of B 1 step down. Each thread still uses its entire row of A and column of B; it simply receives them 1 tile at a time.

5.3 Loading a tile together

A tile holds T^2 values and a block holds T^2 threads, so the loading divides evenly: each thread copies exactly 1 value of each tile. In phase t, thread (ty, tx) copies the value that belongs at its own position:

$$\mathit{tileA}[ty][tx] = a_{\mathit{row},\, tT+tx}, \qquad \mathit{tileB}[ty][tx] = b_{tT+ty,\, \mathit{col}}.$$

As (ty, tx) runs over the block, these assignments fill every position of both tiles exactly once, so no value is copied twice. In Figure 5.3 we watch block $(0, 0)$ at work; each colour belongs to 1 thread, and marks both the value that the thread copies and the sum that it accumulates.

Let us follow thread $(0, 0)$. In phase 0 it copies $a_{00} = 1$ and $b_{00} = 1$, while its 3 neighbours copy the other 6 values. It then multiplies row 0 of *tileA* by column 0 of *tileB*: $1 \cdot 1 + 2 \cdot 0 = 1$. In phase 1 the tiles hold the next pieces, and it adds $3 \cdot 2 + 4 \cdot 1 = 10$, arriving at $c_{00} = 11$.

Where does the saving come from? Look at thread $(0, 1)$ in phase 0 (Figure 5.4). It uses 4 values but copied only 2 of them; its neighbours copied the other 2.

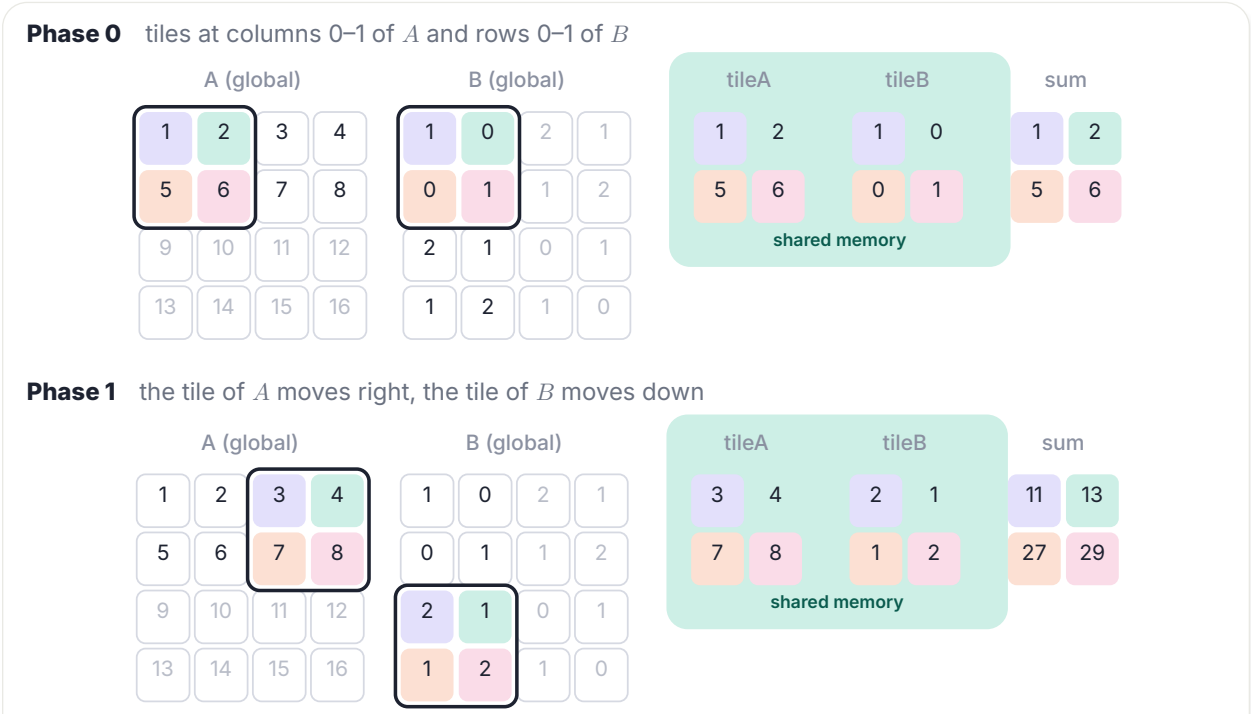


Figure 5.3 Block $(0, 0)$ in 2 phases. Each colour marks 1 thread: the values it copies, the shared-memory slots it fills, and the sum it accumulates.



Figure 5.4 The operands of thread $(0, 1)$ in phase 0. Other threads of the block copied 2 of the 4 operands.

5.4 The algorithm

Each phase repeats the same 4 steps: copy the tiles, wait, compute, wait (Figure 5.5). The 2 waits are barriers, and we explain them in Section 6.

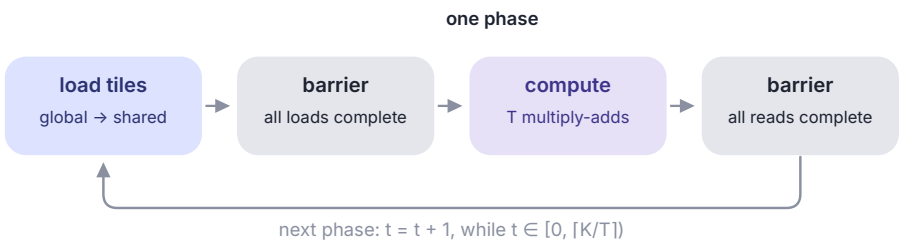


Figure 5.5 1 phase of the tiled kernel. The cycle repeats $\lceil K/T \rceil$ times.

Every thread executes the following procedure. The arrays *tileA* and *tileB* live in shared memory, 1 pair per block.

Tiled-Kernel(A, B, C, M, K, N)

```
1  tx = threadIdx.x,    ty = threadIdx.y
2  row = blockIdx.y · T + ty,    col = blockIdx.x · T + tx
3  sum = 0
4  for t ∈ [0, ⌈K/T⌉) {
5      // copy 1 value of each tile; 0 outside the matrix
6      tileA[ty][tx] = a_{row, tT+tx} if row ∈ [0, M) and tT + tx ∈ [0, K), else 0
7      tileB[ty][tx] = b_{tT+ty, col} if tT + ty ∈ [0, K) and col ∈ [0, N), else 0
8      Barrier()                                   // wait until both tiles are full
9      for u ∈ [0, T) {
10         sum = sum + tileA[ty][u] · tileB[u][tx]
11     }
12     Barrier()                                   // wait until every thread has read the tiles
13 }
14 if row ∈ [0, M) and col ∈ [0, N) {
15     c_{row, col} = sum
16 }
```

Lines 6–7 are the joint copy of Section 5.3, and lines 9–11 add 1 phase of the sum. Lines 8 and 12 are the 2 barriers, and the zeros in lines 6–7 handle matrices whose size is not a multiple of T; Section 6 covers both.

5.5 Counting the fetches again

In each phase, every thread fetches 2 values from global memory, 1 for each tile. Block $(0, 0)$ has 4 threads and runs 2 phases, so it makes $4 \cdot 2 \cdot 2 = 16$ fetches, half of the 32 in the direct kernel. Over all 4 blocks we count 64 fetches instead of 128 (Figure 5.6).

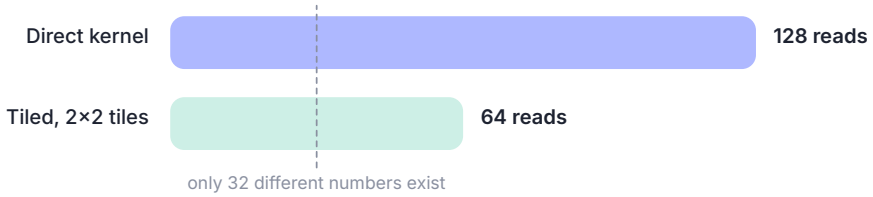


Figure 5.6 Fetches from global memory in the 4×4 example.

In general a block makes $2T^2$ fetches per phase. When T divides every dimension, the grid has $(M/T)(N/T)$ blocks and each block runs K/T phases, so the total is

$$\frac{M}{T} \cdot \frac{N}{T} \cdot \frac{K}{T} \cdot 2T^2 = \frac{2MNK}{T}.$$

Tiling thus divides the traffic to global memory by exactly T: each value is fetched N/T times instead of N times. With $T = 16$, the 1024×1024 product moves 0.54 GB instead of 8.6 GB.

What limits T? A block holds at most 1024 threads, so $T \leq 32$. The 2 tiles take $2T^2$ values of 4 bytes, which for $T = 16$ is only 2 KB of shared memory.

EXERCISES

5.1 Carry out both phases of block $(0, 1)$, which computes c_{02}, c_{03}, c_{12} and c_{13}. Which values does thread $(1, 0)$ copy in each phase?

5.2 With $T = 4$, the whole example is a single block. How many fetches does it make, and how often is each value fetched?

5.3 Let $M = N = K = 100$ and $T = 16$. How many phases does each block run, and what fraction of the values copied in the last phase are zeros?

6 Synchronization

The threads of a block run at different speeds, yet they read and write the same shared-memory slots. Without coordination, 2 things can go wrong: a thread can read a slot before it has been filled, or overwrite a slot that another thread has not finished reading. We now remove the barriers of Tiled-Kernel 1 at a time and watch each error happen on our example.

6.1 The first barrier: copy before computing

Suppose we delete line 8. In phase 1, thread $(0, 0)$ finishes its copying early and starts computing. It needs *tileB*$[1][0]$, which thread $(1, 0)$ is about to fill with $b_{30} = 1$, but thread $(1, 0)$ is running late (Figure 6.1).

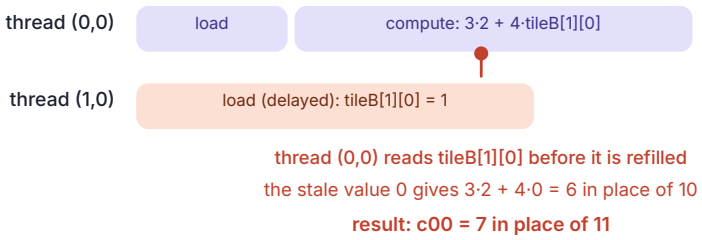


Figure 6.1 Without line 8. Thread $(0, 0)$ reads a slot before thread $(1, 0)$ refills it, and picks up the value left over from phase 0.

The slot still holds 0 from phase 0, and we get $c_{00} = 7$ instead of 11. Line 8 prevents this: no thread may start computing until every thread has finished copying.

6.2 The second barrier: compute before the next copy

Now we restore line 8 and delete line 12 instead. In phase 0, thread $(1, 1)$ finishes its multiply-adds early, moves on to phase 1, and stores $b_{31} = 2$ in *tileB*$[1][1]$. But thread $(0, 1)$ is still in phase 0 and still needs the old value 1 (Figure 6.2).

This time we get $c_{01} = 15$ instead of 13. Line 12 prevents this: no thread may overwrite a tile until every thread has finished reading it. With both barriers in place, every phase follows the same orderly

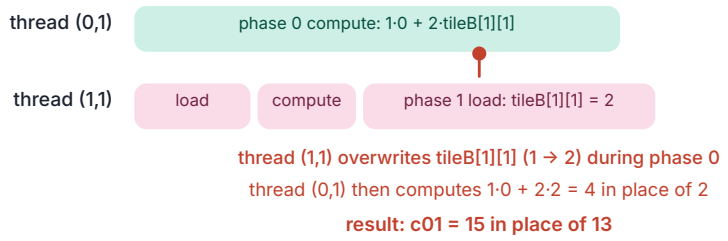


Figure 6.2 Without line 12. Thread $(1, 1)$ overwrites a slot while thread $(0, 1)$ still needs the old value.

schedule (Figure 6.3).

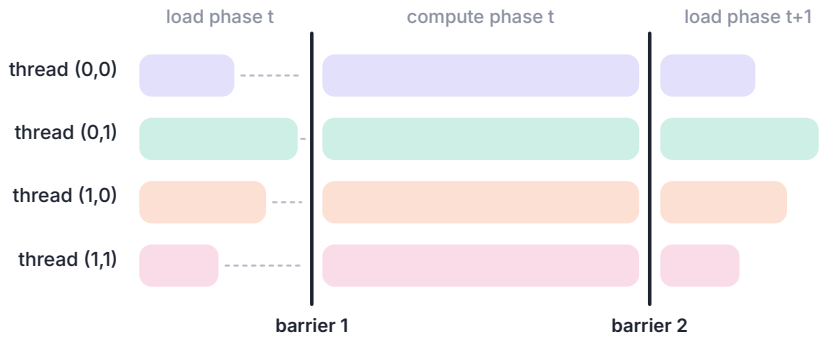


Figure 6.3 1 phase with both barriers. The dotted segments show threads waiting for the slowest thread.

6.3 Correctness

We can now prove that the kernel is correct. The idea is to watch the variable *sum* of 1 thread grow, phase by phase, as in Figure 6.4: before phase t, it holds exactly the first tT terms of its entry of C.

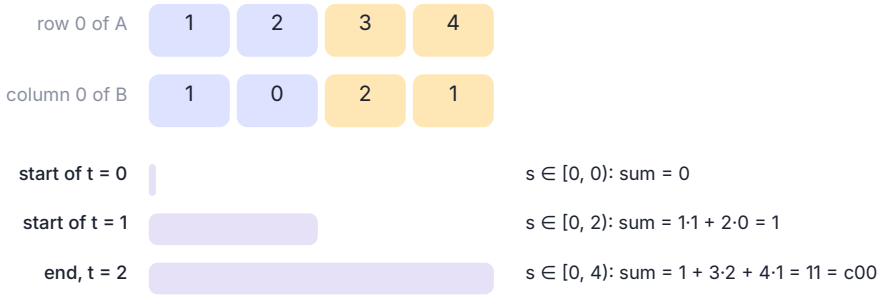


Figure 6.4 The loop invariant for thread $(0, 0)$ with $T = 2$. Before phase t, *sum* covers the terms $s \in [0, tT)$.

Theorem 6.1 When TILED-KERNEL finishes, every entry of C equals the corresponding entry of AB.

Proof. Fix a thread with *row* $\in [0, M)$ and *col* $\in [0, N)$, and treat entries beyond the edge of a matrix as 0. We show that the loop of lines 4–13 maintains the invariant

at the start of phase t, $\quad sum = \sum_{s \in [0,\, tT)} a_{row,\, s}\, b_{s,\, col}$.

Initialization. Before phase 0, $sum = 0$, which is the empty sum.

Maintenance. Assume the invariant holds at the start of phase t. In lines 6–7, the threads of the block fill both tiles completely (Section 5.3). Because of the barrier in line 8, every one of these stores is complete before any thread reaches line 10; this excludes the error of Section 6.1. Hence $tileA[ty][u] = a_{row,\, tT+u}$ and $tileB[u][tx] = b_{tT+u,\, col}$ for every $u \in [0, T)$, and lines 9–11 add exactly the terms with $s \in [tT, tT + T)$. Because of the barrier in line 12, no thread overwrites a tile for phase $t + 1$ while another thread is still reading it in phase t; this excludes the error of Section 6.2. So the invariant holds at the start of phase $t + 1$.

Termination. The loop ends with $t = \lceil K/T \rceil$, so $tT \geq K$. All terms with $s \geq K$ are 0, hence $sum = c_{row,col}$, and line 15 stores it. ∎

6.4 Matrices of any size

Nothing in the algorithm requires the dimensions to be multiples of T. Take C of size 3×3 with $T = 2$. We still need 4 blocks, they stick out past the edge of C, and 7 of their threads have no entry to compute. If also $K = 3$, the second tile of A reaches for a column that does not exist (Figure 6.5).

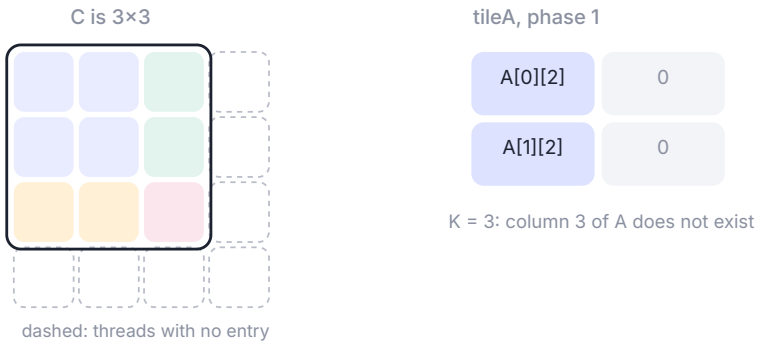


Figure 6.5 Left: 2×2 blocks over a 3×3 matrix; dashed cells are threads with no entry. Right: a tile that extends past the last column of A is filled with zeros.

For a missing value, lines 6–7 store 0, and a product with 0 adds nothing. Threads with no entry need more care, and they explain the only structural difference between our 2 kernels. In the direct kernel such a thread simply returns. In the tiled kernel it must not: a barrier releases the block only when every thread has arrived, and a thread that has returned never arrives, so the rest of the block would wait forever. Instead, these threads stay in the loop, copy zeros, wait at every barrier with everyone else, and skip only the final store in lines 14–16.

RULE

Every thread of a block must reach every barrier. Never place a barrier where some threads of the block can skip it.

EXERCISES

6.1 Without line 8, thread $(1, 1)$ might read *tileA*$[1][0]$ in phase 1 before the slot is refilled. Which value would it use, and what would c_{11} become?

6.2 Explain why the proof of Theorem 6.1 breaks when a thread returns early, even though that thread writes no entry of C.

7 Implementation

We now turn both kernels into Rust. In cuda-oxide, a kernel is an ordinary Rust function marked `#[kernel]`, placed inside a module marked `#[cuda_module]`; the compiler turns the module into GPU code and generates a host-side method with the same name for launching each kernel [4]. We start with the direct kernel, which needs no shared memory, and then add tiling.

7.1 The direct kernel

Both kernels compute positions in row-major memory with the same small helper, the rule Index of Definition 2.2:

```
// Convert (row, col) into an offset in row-major memory.
#[inline]
fn index(row: usize, col: usize, width: usize) -> usize {
    row * width + col
}
```

The direct kernel matches Direct-Kernel of Section 4.1 almost line for line:

```
// C = A x B, with A: M x K, B: K x N and C: M x N.
// One thread computes one element of C, reading straight from global memory.
#[kernel]
pub fn matmul_naive(m: u32, n: u32, k: u32, a: &[f32], b: &[f32], c: &mut [f32]) {
    let (m, n, k) = (m as usize, n as usize, k as usize);
    let row = thread::index_2d_row();
    let col = thread::index_2d_col();
    if row ⩾ m || col ⩾ n {
        return; // this thread lies outside C
    }

    let mut sum = 0.0f32;
    for i in 0..k {
        sum += a[index(row, i, k)] * b[index(i, col, n)];
    }
    c[index(row, col, n)] = sum;
}
```

The sizes arrive as `u32`, the type of a kernel argument, and we convert them to `usize` once, so that all index arithmetic uses a single type. `index_2d_row()` and `index_2d_col()` are lines 1–2 of the pseudocode, the early `return` is lines 3–5, and the loop over `i` is lines 7–9. The kernel needs no `unsafe`: it

touches only A, B and C through ordinary slices.

7.2 Shared arrays

In cuda-oxide we declare an array in shared memory as a `static mut` of type `SharedArray<T, N>` inside the kernel [4]. Its length is part of its type and must be fixed at compile time, so the tile size is the only constant in our program:

```
const TILE: usize = 16;
```

The sizes M, K and N remain ordinary run-time values and need not be multiples of 16. We store each tile as a flat array of T^2 values in row-major order, so `index(ty, tx, TILE)` finds slot (ty, tx) with the rule of Definition 2.2 (Figure 7.1). Every access to a shared array must be marked `unsafe`, because the compiler cannot check how concurrent threads use it; Section 6 supplies that argument.

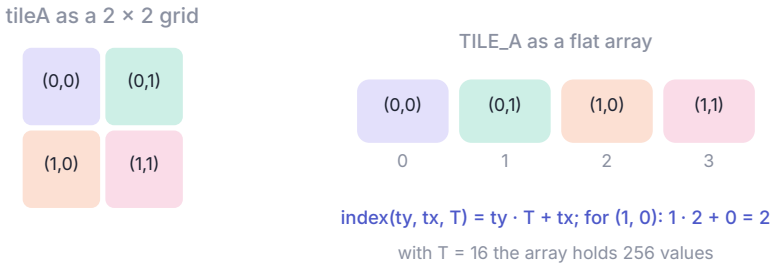


Figure 7.1 A 2×2 tile stored as a flat array of 4 slots. Slot $(1, 0)$ is at $1 \cdot 2 + 0 = 2$.

7.3 The tiled kernel

Before reading the code, it helps to trace where the data of 1 thread travels (Figure 7.2): each phase, 2 values go from global memory into shared memory; from there, $2T$ values feed the multiply-adds; at the end, a single store writes *sum* into C.

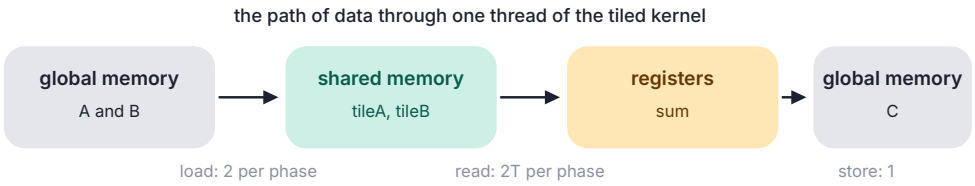


Figure 7.2 The path of data through 1 thread of the tiled kernel.

```
#[kernel]
pub fn matmul_tiled(m: u32, n: u32, k: u32, a: &[f32], b: &[f32], c: &mut [f32]) {
    static mut TILE_A: SharedArray<f32, { TILE * TILE }> = SharedArray::UNINIT;
    static mut TILE_B: SharedArray<f32, { TILE * TILE }> = SharedArray::UNINIT;

    let (m, n, k) = (m as usize, n as usize, k as usize);
```

```
    let row = thread::index_2d_row();
    let col = thread::index_2d_col();
    let tx = thread::threadIdx_x() as usize;
    let ty = thread::threadIdx_y() as usize;

    let mut sum = 0.0f32;
    for t in 0..k.div_ceil(TILE) {
        let a_col = t * TILE + tx;
        let b_row = t * TILE + ty;

        // Load: each thread copies one element of A and one of B.
        // Positions past the edge of a matrix are filled with zero.
        // SAFETY: each thread writes only its own slot (ty, tx) of each tile,
        // and the barrier below completes these writes before any thread reads.
        unsafe {
            TILE_A[index(ty, tx, TILE)] =
                if row < m && a_col < k { a[index(row, a_col, k)] } else { 0.0 };
            TILE_B[index(ty, tx, TILE)] =
                if b_row < k && col < n { b[index(b_row, col, n)] } else { 0.0 };
        }
        thread::sync_threads(); // wait until both tiles are full

        // Compute: one partial dot product, entirely from shared memory.
        for i in 0..TILE {
            // SAFETY: the tiles are only read here, between the 2 barriers,
            // so no thread writes them at the same time.
            unsafe {
                sum += TILE_A[index(ty, i, TILE)] * TILE_B[index(i, tx, TILE)];
            }
        }
        thread::sync_threads(); // wait until every thread has read the tiles
    }

    if row < m && col < n {
        c[index(row, col, n)] = sum;
    }
}
```

The code matches TILED-KERNEL line by line. The loop over `t` is line 4, and `a_col` and `b_row` are the indices $tT + tx$ and $tT + ty$ of lines 6–7. The 2 calls to `sync_threads()` are the barriers of lines 8 and 12; both sit outside every `if`, so every thread reaches them. The final `if` corresponds to lines 14–16.

Each `unsafe` block carries a `// SAFETY:` comment, the Rust convention for stating why the block is sound. The 2 comments summarize Section 6: every thread writes only its own slot, and the barriers keep writes and reads of the tiles apart.

7.4 The host program

On the host side, we create the inputs, copy them to the device, launch each kernel with the same configuration, copy each result back, and compare it with a product computed on the CPU. The configuration is the 2-dimensional grid of Section 3.2. For $M = 128$ and $N = 96$ it consists of 6×8 blocks of 16×16 threads (Figure 7.3).

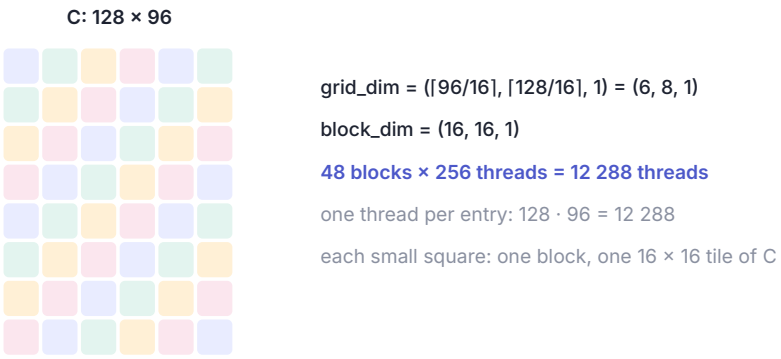


Figure 7.3 The launch for C of size 128×96 with $T = 16$: 48 blocks, 12 288 threads, 1 thread per entry of C.

```
let side = TILE as u32;
let config = LaunchConfig {
    grid_dim: ((n as u32).div_ceil(side), (m as u32).div_ceil(side), 1),
    block_dim: (side, side, 1),
    shared_mem_bytes: 0,
};
```

We set `shared_mem_bytes` to 0 because our tiles are fixed-size arrays declared inside the kernel; the field only reserves shared memory whose size is chosen at launch. The launch itself goes through the method that `#[cuda_module]` generated for each kernel:

```
let mut c_gpu = DeviceBuffer::<f32>::zeroed(&stream, m * n)?;

// SAFETY: the grid covers C, each thread writes only its own (row, col),
// and the buffers hold M*K, K*N and M*N elements.
unsafe {
    module.matmul_tiled(&stream, config, m as u32, n as u32, k as u32, &a_gpu, &b_gpu, &mut c_gpu)?;
}

let result = c_gpu.to_host_vec(&stream)?; // waits for the kernel to finish
```

A launch is `unsafe` because the compiler cannot check what the kernel does with the buffers; the `SAFETY` comment records the 3 facts that make this launch sound. Launching `matmul_naive` looks the same.

Exact verification. We fill the inputs with integers in $[-5, 5]$. Every product is then at most 25 in magnitude, and every partial sum at most $25K = 1600$ for $K = 64$. Single-precision floating point represents every integer up to 2^{24} exactly, so the GPU and the CPU must produce identical results whatever order they add in, and any difference at all signals a bug.

Timing. A kernel launch returns as soon as the work is queued, before the GPU has done it. Copying C back has to wait for the kernel, so the time we report covers the kernel plus the copy. At $128 \times 64 \times 96$, both kernels finish within microseconds and fixed overheads dominate; the difference between the kernels becomes visible at sizes such as $M = N = K = 2048$.

EXERCISES

7.1 Set `TILE` to 32. Predict the effect on fetches from global memory and on shared memory per block.

7.2 Each thread of the tiled kernel performs 2 arithmetic operations for every 2 values it reads from shared memory. Design a variant in which each thread computes several entries of C, so that every value read from shared memory is used more than once.

8 Measurements

8.1 Platform and method

We ran both kernels on an NVIDIA Jetson AGX Thor developer kit. The module combines an Arm CPU with a Blackwell GPU of 2560 CUDA cores, for which we compiled to the architecture `sm_110a`. CPU and GPU share 128 GB of LPDDR5X memory with a peak bandwidth of 273 GB/s [6]. We multiplied matrices with $M = N = K = 512$, using $T = 16$ and blocks of 16×16 threads. We timed the kernels with CUDA events and report the mean of 5 launches; we timed the copies once, with the host clock. For each result, 182 sampled entries matched the CPU product exactly.

8.2 Results

Kernel	Time	Throughput	Fetches from global memory
Direct	1.158 ms	231.8 GFLOP/s	1024 MiB
Tiled	0.285 ms	941.5 GFLOP/s	64 MiB
Ratio	4.06	4.06	16

We count $2MNK = 2.68 \times 10^8$ floating-point operations per product. The fetch volumes follow from Section 5.5: the direct kernel fetches $2MNK$ values of 4 bytes, or 1024 MiB, and the tiled kernel $1/16$ of that, or 64 MiB (Figure 8.1).

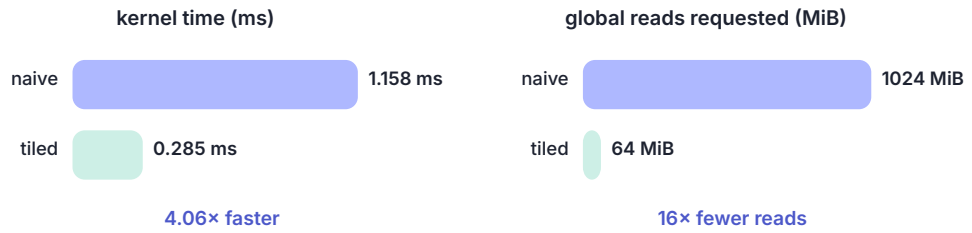


Figure 8.1 Measured kernel time and modelled fetch volume for $M = N = K = 512$ on Jetson AGX Thor.

8.3 Why the speedup is 4, not 16

Our model in Section 5.5 assumed that every fetch goes all the way to DRAM. Our measurement shows that this cannot be true for the direct kernel. Fetching 1024 MiB in 1.158 ms would require 927 GB/s, which is 3.4 times the DRAM peak of 273 GB/s (Figure 8.2). Since DRAM delivers at most 273 GB/s, at least $1 - 273/927 = 70\%$ of the direct kernel's fetches must have been answered by the

L1 and L2 caches. In other words, many of the repeated fetches of Figure 4.2 find their value already in a cache, left there by a neighbouring thread.

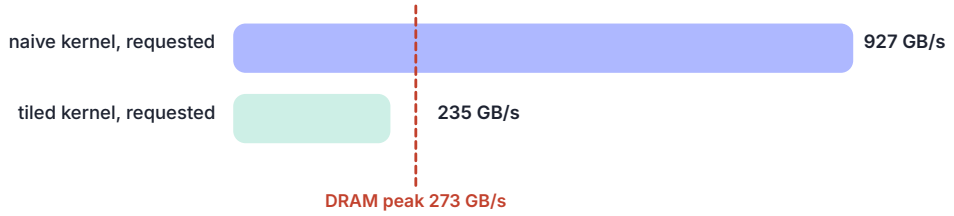


Figure 8.2 Fetch rate of each kernel: fetch volume divided by kernel time. The direct kernel asks for 3.4 times more than DRAM can deliver, so caches must have answered most of its fetches.

So the hardware caches already recover part of the reuse that tiling makes explicit, and the measured gain is 4.06 rather than 16. The difference between the 2 approaches is control. A value stays in a cache only by chance, and that chance shrinks as the matrices grow beyond the cache. A value in shared memory stays there for exactly 1 phase, by construction, at every matrix size. The tiled kernel needs 235 GB/s, comfortably below the DRAM peak.

8.4 Copies between host and device

Copying A and B to the device took 2.944 ms, and copying C back took 0.048 ms. Together the copies took 2.992 ms, longer than either kernel. We measured the first copy only once, so it includes any one-time cost of a first transfer. On Jetson AGX Thor, host and device share a single physical memory, so these copies move data from 1 part of the same DRAM to another. Appendix B gives a variant without the copies of A and B; we leave the comparison as Exercise 8.1.

EXERCISES

8.1 Benchmark the unified-memory variant of Appendix B against the copy-based program of Appendix A, on the same machine, with the same timer and 5 runs each, for $M = N = K \in \{512, 1024, 2048\}$. At which sizes, if any, does avoiding the copies of A and B reduce the total time?

9 Conclusion

The tiled kernel differs from the direct kernel in 3 ways: it stages values in shared memory, it splits each dot product into phases of length T, and it uses 2 barriers to keep copying and computing apart. Staging divides global memory traffic by T, and the barriers make the staging correct. On Jetson AGX Thor, the tiled kernel ran 4.06 times faster than the direct kernel at 512×512; the gain is below 16 because the caches already absorb part of the direct kernel's redundant fetches. The same cycle of copy, wait, compute, wait underlies faster kernels as well, in which each thread computes several entries of C and tiles are staged at several levels of the memory hierarchy [2, 5].

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[2] W. W. Hwu, D. B. Kirk and I. El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*, 4th ed. Morgan Kaufmann, 2022.

[3] NVIDIA Corporation. *CUDA C++ Programming Guide*.

[4] NVIDIA Labs. *The cuda-oxide Book*. `https://nvlabs.github.io/cuda-oxide/`

[5] V. Volkov and J. W. Demmel. Benchmarking GPUs to tune dense linear algebra. In *Proceedings of the ACM/IEEE Conference on Supercomputing (SC08)*, 2008.

[6] JetsonHacks. NVIDIA Jetson AGX Thor: the new king of embedded. 2025. `https://jetsonhacks.com/2025/09/21/nvidia-jetson-agx-thor-the-new-king-of-embedded/`

A Complete source

The file `src/main.rs`, with both kernels and the host program.

```rust
use cuda_core::simt::LaunchConfig;
use cuda_core::{CudaContext, CudaStream, DeviceBuffer};
use cuda_device::{kernel, thread, SharedArray};
use cuda_host::cuda_module;
use std::time::Instant;

/// Edge length of a tile. Shared arrays are sized at compile time,
/// so this is the one constant; every matrix dimension stays dynamic.
const TILE: usize = 16;

// Convert (row, col) into an offset in row-major memory.
#[inline]
fn index(row: usize, col: usize, width: usize) -> usize {
    row * width + col
}

#[cuda_module]
mod kernels {
    use super::*;

    // C = A x B, with A: M x K, B: K x N and C: M x N.
    // One thread computes one element of C, reading straight from global memory.
    #[kernel]
    pub fn matmul_naive(m: u32, n: u32, k: u32, a: &[f32], b: &[f32], c: &mut [f32]) {
        let (m, n, k) = (m as usize, n as usize, k as usize);
        let row = thread::index_2d_row();
        let col = thread::index_2d_col();
        if row >= m || col >= n {
            return;
        }

        let mut sum = 0.0f32;
        for i in 0..k {
            sum += a[index(row, i, k)] * b[index(i, col, n)];
        }
        c[index(row, col, n)] = sum;
    }

    // Same result, but each block stages TILE x TILE pieces of A and B
    // in shared memory and every thread of the block reuses them.
```

```
    #[kernel]
    pub fn matmul_tiled(m: u32, n: u32, k: u32, a: &[f32], b: &[f32], c: &mut [f32]) {
        static mut TILE_A: SharedArray<f32, { TILE * TILE }> = SharedArray::UNINIT;
        static mut TILE_B: SharedArray<f32, { TILE * TILE }> = SharedArray::UNINIT;

        let (m, n, k) = (m as usize, n as usize, k as usize);
        let row = thread::index_2d_row();
        let col = thread::index_2d_col();
        let tx = thread::threadIdx_x() as usize;
        let ty = thread::threadIdx_y() as usize;

        let mut sum = 0.0f32;
        for t in 0..k.div_ceil(TILE) {
            let a_col = t * TILE + tx;
            let b_row = t * TILE + ty;

            // Load: each thread copies one element of A and one of B.
            // Positions past the edge of a matrix are filled with zero.
            // SAFETY: each thread writes only its own slot (ty, tx) of each tile,
            // and the barrier below completes these writes before any thread reads.
            unsafe {
                TILE_A[index(ty, tx, TILE)] =
                    if row < m && a_col < k { a[index(row, a_col, k)] } else { 0.0 };
                TILE_B[index(ty, tx, TILE)] =
                    if b_row < k && col < n { b[index(b_row, col, n)] } else { 0.0 };
            }
            thread::sync_threads(); // wait until both tiles are full

            // Compute: one partial dot product, entirely from shared memory.
            for i in 0..TILE {
                // SAFETY: the tiles are only read here, between the 2 barriers,
                // so no thread writes them at the same time.
                unsafe {
                    sum += TILE_A[index(ty, i, TILE)] * TILE_B[index(i, tx, TILE)];
                }
            }
            thread::sync_threads(); // wait until every thread has read the tiles
        }

        if row < m && col < n {
            c[index(row, col, n)] = sum;
        }
    }
}

type MayBeError = Box<dyn std::error::Error>;

fn main() -> Result<(), MayBeError> {
    let ctx = CudaContext::new(0)?;
    let stream = ctx.default_stream();
    let module = kernels::load(&ctx)?;

    // Any sizes work; none of them has to be a multiple of TILE.
    let (m, k, n) = (128, 64, 96);

    let (a_host, b_host) = inputs(m, k, n);
    let a_gpu = DeviceBuffer::from_host(&stream, &a_host)?;
    let b_gpu = DeviceBuffer::from_host(&stream, &b_host)?;

    // A 2D grid over C: x runs along columns (N), y along rows (M).
    let side = TILE as u32;
    let config = LaunchConfig {
        grid_dim: ((n as u32).div_ceil(side), (m as u32).div_ceil(side), 1),
        block_dim: (side, side, 1),
        shared_mem_bytes: 0,
    };
```

```
    for tiled in [false, true] {
        let mut c_gpu = DeviceBuffer::<f32>::zeroed(&stream, m * n)?;
        let start = Instant::now();

        // SAFETY: the grid covers C, each thread writes only its own (row, col),
        // and the buffers hold M*K, K*N and M*N elements.
        unsafe {
            if tiled {
                module.matmul_tiled(
                    &stream, config, m as u32, n as u32, k as u32, &a_gpu, &b_gpu, &mut c_gpu,
                )?;
            } else {
                module.matmul_naive(
                    &stream, config, m as u32, n as u32, k as u32, &a_gpu, &b_gpu, &mut c_gpu,
                )?;
            }
        }

        // Copying back waits for the kernel, so this time includes its execution.
        let result = c_gpu.to_host_vec(&stream)?;
        let ms = start.elapsed().as_secs_f64() * 1e3;
        let label = if tiled { "tiled" } else { "naive" };
        println!("{label}: {ms:.3} ms, kernel plus copy back");

        verify(&a_host, &b_host, &result, (m, k, n));
        println!("{label}: GPU matches CPU on all {} elements", m * n);
        print_corner(&result, n, 4);
    }

    Ok(())
}

// A (M x K) and B (K x N), row-major and generated deterministically.
// Every value is a small integer in [-5, 5], so all products and partial
// sums are exactly representable and the CPU check is exact.
fn inputs(m: usize, k: usize, n: usize) -> (Vec<f32>, Vec<f32>) {
    let generate = |rows: usize, cols: usize, seed: usize| -> Vec<f32> {
        (0..rows * cols)
            .map(|i| {
                let (row, col) = (i / cols, i % cols);
                ((row * 13 + col * 7 + seed) % 11) as f32 - 5.0
            })
            .collect()
    };
    (generate(m, k, 0), generate(k, n, 5))
}

// Check every element against a plain CPU implementation.
fn verify(a: &[f32], b: &[f32], c: &[f32], (m, k, n): (usize, usize, usize)) {
    for row in 0..m {
        for col in 0..n {
            let expected: f32 = (0..k)
                .map(|i| a[index(row, i, k)] * b[index(i, col, n)])
                .sum();
            let actual = c[index(row, col, n)];

            assert!(
                (actual - expected).abs() < 1e-5,
                "Mismatch at ({row}, {col}): got {actual}, want {expected}"
            );
        }
    }
}

// The full C is too wide to read in a terminal, so show its top-left corner.
```

```
fn print_corner(values: &[f32], width: usize, size: usize) {
    let size = size.min(values.len() / width).min(width);

    println!("C[0..{size}][0..{size}]:");
    for row in 0..size {
        let start = row * width;
        let cells: Vec<String> = values[start..start + size]
            .iter()
            .map(|value| format!("{value:>8.1}"))
            .collect();
        println!("  {}", cells.join(" "));
    }
}
```

B Unified-memory variant

On Jetson AGX Thor, CPU and GPU share a single physical memory, so the GPU can access host memory without a copy. In the variant below, we pass A and B to the tiled kernel as closures over host slices, `move |i| a[i]`. Through Heterogeneous Memory Management [4], the GPU then reads the host slices in place, and A and B are never copied. C is still written to a device buffer and copied back. A kernel with closure parameters is generic, so we load its module by name with `load_kernel_module` and wrap it with `kernels::from_module`. The program checks 4 entries of C against the CPU; benchmarking is left as Exercise 8.1.

```
// Tiled C = A x B where the GPU reads A and B straight from host memory.
// On Jetson AGX Thor, CPU and GPU share 1 physical memory, so A and B are
// never copied. Only C (1 MiB) is copied back.

use cuda_core::simt::LaunchConfig;
use cuda_core::{CudaContext, DeviceBuffer};
use cuda_device::{kernel, thread, SharedArray};
use cuda_host::{cuda_module, load_kernel_module};
use std::time::Instant;

const TILE: usize = 16;

#[inline]
fn index(row: usize, col: usize, width: usize) -> usize {
    row * width + col
}

#[cuda_module]
mod kernels {
    use super::*;

    // The tiled kernel, except A and B are closures over host slices.
    #[kernel]
    pub fn matmul_unified<FA: Fn(usize) -> f32 + Copy, FB: Fn(usize) -> f32 + Copy>(
        m: u32, n: u32, k: u32, a: FA, b: FB, c: &mut [f32],
    ) {
        static mut TILE_A: SharedArray<f32, { TILE * TILE }> = SharedArray::UNINIT;
        static mut TILE_B: SharedArray<f32, { TILE * TILE }> = SharedArray::UNINIT;

        let (m, n, k) = (m as usize, n as usize, k as usize);
        let row = thread::index_2d_row();
        let col = thread::index_2d_col();
        let tx = thread::threadIdx_x() as usize;
```

```
        let ty = thread::threadIdx_y() as usize;

        let mut sum = 0.0f32;
        for t in 0..k.div_ceil(TILE) {
            let (a_col, b_row) = (t * TILE + tx, t * TILE + ty);
            // SAFETY: each thread writes only its own slot (ty, tx) of each tile,
            // and the barrier below completes these writes before any thread reads.
            unsafe {
                TILE_A[index(ty, tx, TILE)] = if row < m && a_col < k { a(index(row, a_col, k)) } else { 0.0 };
                TILE_B[index(ty, tx, TILE)] = if b_row < k && col < n { b(index(b_row, col, n)) } else { 0.0 };
            }
            thread::sync_threads(); // wait until both tiles are full

            for i in 0..TILE {
                // SAFETY: the tiles are only read here, between the 2 barriers,
                // so no thread writes them at the same time.
                unsafe { sum += TILE_A[index(ty, i, TILE)] * TILE_B[index(i, tx, TILE)]; }
            }
            thread::sync_threads(); // wait until every thread has read the tiles
        }

        if row < m && col < n {
            c[index(row, col, n)] = sum;
        }
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let ctx = CudaContext::new(0)?;
    let stream = ctx.default_stream();
    // A kernel that takes closures is loaded by name, then wrapped.
    let module = load_kernel_module(&ctx, "tiled_matrix_mul_unified_memory")
        .map_err(|e| format!("load: {e}"))?;
    let module = kernels::from_module(module).map_err(|e| format!("from_module: {e}"))?;

    let n = 512; // square: M = K = N
    let a_host: Vec<f32> = (0..n * n).map(|i| ((i * 7) % 11) as f32 - 5.0).collect();
    let b_host: Vec<f32> = (0..n * n).map(|i| ((i * 13 + 5) % 11) as f32 - 5.0).collect();
    let (a, b): (&[f32], &[f32]) = (&a_host, &b_host);

    let side = TILE as u32;
    let config = LaunchConfig {
        grid_dim: ((n as u32).div_ceil(side), (n as u32).div_ceil(side), 1),
        block_dim: (side, side, 1),
        shared_mem_bytes: 0,
    };

    let run = || -> Result<Vec<f32>, Box<dyn std::error::Error>> {
        let mut c_gpu = DeviceBuffer::<f32>::zeroed(&stream, n * n)?;
        // SAFETY: the grid covers C, each thread writes only its own entry,
        // and a_host and b_host outlive the kernel: to_host_vec below waits for it.
        unsafe {
            module.matmul_unified::<_, _>(
                stream.as_ref(), config, n as u32, n as u32, n as u32,
                move |i: usize| a[i], move |i: usize| b[i], &mut c_gpu,
            )?;
        }
        Ok(c_gpu.to_host_vec(&stream)?) // waits for the kernel
    };

    run()?; // warm-up
    let start = Instant::now();
    let c = run()?;
    println!("unified: {:.3} ms end to end (kernel + copy C back), {n}x{n}", start.elapsed().as_secs_f64() * 1e3);

    for (row, col) in [(0, 0), (1, 2), (n / 2, n / 3), (n - 1, n - 1)] {
```

```
        let want: f32 = (0..n).map(|s| a[row * n + s] * b[s * n + col]).sum();
        assert_eq!(c[row * n + col], want, "mismatch at ({row}, {col})");
    }
    println!("PASSED");
    Ok(())
}
```